

IMPLEMENTATION_PLAN

HelixCode Feature Implementation Plan

Version: 1.0 **Date:** 2025-11-05 **Timeline:** 10 weeks (parallelized)

Implementation Strategy

This plan implements all identified features from the comprehensive analysis in a structured, parallelized approach. Each feature will include: - Production code - Comprehensive tests (unit + integration) - Documentation - User manual entries - API documentation

Phase 1: Critical Tools & Infrastructure (Weeks 1-2)

1.1 File System Tools Package

Location: internal/tools/filesystem/ **Files to Create:** - reader.go - File reading with glob patterns - writer.go - File writing with atomic operations - editor.go - In-place file editing with diff support - searcher.go - File content search (grep-like) - filesystem_test.go - Comprehensive tests

Features: - Read single/multiple files - Write with backup/rollback - Edit with search/replace - Recursive directory operations - Git-aware file filtering - Permission checks - Symlink handling

Tests: 20+ test cases covering all operations

Documentation: - docs/tools/filesystem.md - Complete API reference - Examples for each operation - Security considerations

1.2 Shell Execution Package

Location: internal/tools/shell/ **Files to Create:** - executor.go - Command execution with safety checks - sandbox.go - Sandboxed execution environment - output.go - Output streaming and capture - shell_test.go - Tests including security scenarios

Features: - Safe command execution with allowlist/blocklist - Real-time output streaming - Timeout management - Environment variable isolation - Working directory control - Signal handling (SIGINT, SIGTERM) - Command history logging - Dry-run mode

Tests: 15+ test cases including security tests

Documentation: - docs/tools/shell.md - Usage guide - Security best practices - Configuration options

1.3 Browser Control Package

Location: internal/tools/browser/ **Files to Create:** - controller.go - Puppeteer/chromedp integration - actions.go - Browser actions (click, type, scroll, screenshot) - discovery.go - Chrome/Chromium detection - browser_test.go - Headless browser tests

Features: - Launch/attach to Chrome/Chromium - Actions: launch, click, type, scroll, screenshot, close - Screenshot with coordinate annotation - Console log capture - Page navigation - Element selection - Headless/headed modes - Connection management

Tests: 12+ test cases with mock browser

Documentation: - docs/tools/browser.md - Complete guide - Computer Use integration - Troubleshooting guide

1.4 Codebase Mapping Package

Location: internal/tools/mapping/ **Files to Create:** - mapper.go - Main mapping engine - treesitter.go - Tree-sitter integration - cache.go - Disk cache for parsed results - languages.go - Language-specific queries - mapping_test.go - Parser tests

Features: - Tree-sitter based AST parsing - Support for 30+ languages (Go, TypeScript, Python, Rust, Java, C++, etc.) - Disk cache (.helix.cache/) - Token-based context sizing - Relative indentation for fuzzy matching - Function/class/method extraction - Import/dependency analysis

Tests: 25+ test cases with sample code in multiple languages

Documentation: - docs/tools/codebase-mapping.md - Architecture guide - Supported languages - Cache management

1.5 Multi-File Editing Package

Location: `internal/tools/multiedit/` **Files to Create:** - `editor.go` - Multi-file atomic editing - `transaction.go` - Transaction-based edits - `diff.go` - Diff generation and application - `multiedit_test.go` - Atomic operation tests

Features: - Atomic multi-file edits (all or nothing) - Transaction-based with rollback - Unified diff generation - Conflict detection - Backup before edit - Git integration - Preview mode

Tests: 18+ test cases including rollback scenarios

Documentation: - `docs/tools/multi-file-editing.md` - Usage guide - Transaction management - Error recovery

Phase 2: Workflow Enhancements (Weeks 2-3)

2.1 Plan Mode Implementation

Location: `internal/workflow/planmode/` **Files to Create:** - `planner.go` - Two-phase planning system - `options.go` - Option presentation and selection - `executor.go` - Plan execution - `planmode_test.go` - Workflow tests

Features: - Two-phase workflow (Plan → Act) - Structured option presentation - User selection interface - YOLO auto-execution mode - Progress tracking - Mode switching - Task breakdown

Tests: 15+ test cases covering full workflow

Documentation: - `docs/workflows/plan-mode.md` - Complete guide - Best practices - Example workflows

2.2 Auto-Commit System

Location: `internal/tools/git/` **Files to Create:** - `autocommit.go` - Intelligent commit system - `message_generator.go` - LLM-powered commit messages - `attribution.go` - Co-author attribution - `git_test.go` - Git operation tests

Features: - LLM-generated commit messages - Diff analysis for semantic commits - Co-author attribution - Multi-language commit messages - Conventional commits support - Amend detection - Pre-commit hook integration

Tests: 12+ test cases with mock git repo

Documentation: - docs/tools/auto-commit.md - Usage guide - Commit message generation - Attribution options

2.3 Context Compression

Location: internal/llm/compression/ **Files to Create:** - compressor.go - Conversation summarization - strategies.go - Compression strategies - retention.go - Message retention policies - compression_test.go - Compression tests

Features: - Automatic history summarization - Token-based thresholds - Sliding window retention - Semantic preservation - /compress command - Configurable policies

Tests: 10+ test cases with mock conversations

Documentation: - docs/features/context-compression.md - Guide - Compression strategies - Configuration

Phase 3: Advanced Providers (Weeks 3-4)

3.1 AWS Bedrock Provider

Location: internal/llm/bedrock_provider.go **Tests:** internal/llm/bedrock_provider_test.go

Models: - Claude 4 Sonnet/Opus (via Bedrock) - Claude 3.5/3.7 Sonnet - Titan, Jurassic, Command models

Features: - AWS SDK v2 integration - IAM authentication - Cross-region inference - Streaming support - Model invocation via Bedrock runtime

Tests: 15+ test cases with mock AWS API

Documentation: - docs/providers/bedrock.md - Setup guide - IAM configuration - Model availability by region

3.2 Azure OpenAI Provider

Location: internal/llm/azure_provider.go **Tests:** internal/llm/azure_provider_test.go

Models: - All OpenAI models via Azure - Multiple deployment support - Region-specific endpoints

Features: - Microsoft Entra ID authentication - API key authentication - Deployment-based routing - API version management - Streaming support

Tests: 15+ test cases with mock Azure API

Documentation: - docs/providers/azure.md - Setup guide - Authentication methods - Deployment configuration

3.3 VertexAI Provider

Location: internal/llm/vertexai_provider.go **Tests:** internal/llm/vertexai_provider_test.go

Models: - Gemini models via VertexAI - Claude via VertexAI (Model Garden) - PaLM 2 models

Features: - Google Cloud authentication - Service account support - Project/location-based routing - Streaming support

Tests: 15+ test cases with mock GCP API

Documentation: - docs/providers/vertexai.md - Setup guide - GCP authentication - Project configuration

3.4 Groq Provider

Location: internal/llm/groq_provider.go **Tests:** internal/llm/groq_provider_test.go

Models: - Llama 3.3 70B - Mixtral 8x7B - Ultra-fast inference

Features: - Simple API key authentication - Extremely low latency - High throughput - OpenAI-compatible API

Tests: 12+ test cases

Documentation: - docs/providers/groq.md - Quick start - Performance characteristics

Phase 4: Enhanced Tools (Weeks 4-5)

4.1 Web Tools Package

Location: internal/tools/web/ **Files to Create:** - search.go - Web search integration (Google, Bing, DuckDuckGo) - fetch.go - HTTP fetching with caching - parser.go - HTML/markdown conversion - web_test.go - HTTP mock tests

Features: - Web search with multiple engines - URL fetching with proxy support - HTML to markdown conversion - Caching (15-minute TTL) - Rate limiting - User-agent rotation

Tests: 15+ test cases with mock HTTP

Documentation: - docs/tools/web.md - Usage guide - Search engines - Caching policy

4.2 Tool Confirmation System

Location: internal/tools/confirmation/ **Files to Create:** - confirmer.go - Interactive confirmation - policies.go - Approval policies - audit.go - Audit logging - confirmation_test.go - Policy tests

Features: - Interactive yes/no/always/never prompts - Dangerous operation detection - Policy-based auto-approval - Audit logging - Confirmation levels (info, warning, danger) - Batch approval mode

Tests: 10+ test cases

Documentation: - docs/tools/confirmation.md - Configuration - Policy system - Audit logs

4.3 Voice-to-Code

Location: internal/tools/voice/ **Files to Create:** - recorder.go - Audio recording - transcriber.go - Whisper API integration - device.go - Audio device management - voice_test.go - Mock audio tests

Features: - Audio recording from microphone - Device selection - Whisper transcription - Language support - Volume level detection - WAV/MP3 format

Tests: 8+ test cases with mock audio

Documentation: - docs/tools/voice.md - Setup guide - Device configuration - Language support

Phase 5: Advanced Features (Weeks 5-6)

5.1 Checkpoint Snapshots

Location: internal/workflow/snapshots/ **Files to Create:** - snapshot.go - Workspace snapshots - comparison.go - Diff between snapshots - restore.go - Rollback to snapshot - snapshots_test.go - Snapshot tests

Features: - Git-based workspace snapshots - Compare any two snapshots - Restore to specific snapshot - Automatic snapshot on task steps - Snapshot metadata (timestamp, task, status)

Tests: 12+ test cases

Documentation: - docs/features/snapshots.md - Usage guide - Snapshot management - Restore procedures

5.2 Autonomy Modes

Location: internal/workflow/autonomy/ **Files to Create:** - modes.go - 5 autonomy levels - controller.go - Mode switching - config.go - Per-mode configuration - autonomy_test.go - Mode tests

Modes: 1. **Full Auto:** Complete automation 2. **Semi Auto:** Balanced (auto context, manual apply) 3. **Basic Plus:** Smart semi-automation 4. **Basic:** Manual workflow 5. **None:** Step-by-step control

Tests: 10+ test cases

Documentation: - docs/features/autonomy-modes.md - Mode guide - Configuration - Best practices

5.3 Vision Auto-Switching

Location: internal/llm/vision/ **Files to Create:** - detector.go - Image detection - switcher.go - Auto model switching - config.go - Switch modes (once, session, persist) - vision_test.go - Detection tests

Features: - Detect images in input - Auto-switch to vision models - User confirmation - Switch modes: once (one-time), session (this session), persist (always) - Model capability checking

Tests: 8+ test cases

Documentation: - docs/features/vision-auto-switch.md - Guide - Configuration - Supported models

Phase 6: Documentation & Polish (Weeks 6-7)

6.1 Comprehensive User Manual

Location: docs/USER_MANUAL.md

Sections: 1. Getting Started 2. Configuration 3. LLM Providers (all 14+) 4. Tools Reference (all tools) 5. Workflows (Plan Mode, etc.) 6. Advanced Features 7. Troubleshooting 8. FAQ

6.2 API Documentation

Location: docs/API_REFERENCE.md

Sections: 1. REST API Endpoints 2. WebSocket Protocol 3. MCP Protocol 4. Authentication 5. Request/Response Formats 6. Error Handling 7. Rate Limiting 8. Examples (curl, Go, Python, JavaScript)

6.3 Feature Documentation

Location: docs/features/

Files (one per feature): - extended-thinking.md - prompt-caching.md - plan-mode.md - browser-control.md - codebase-mapping.md - auto-commit.md - context-compression.md - voice-to-code.md - snapshots.md - autonomy-modes.md - vision-auto-switch.md

Testing Strategy

Unit Tests

- Each package has *_test.go with 10-25 test cases
- Mock external dependencies (HTTP, file system, git, etc.)
- Test error conditions
- Test edge cases
- Target: 90%+ coverage

Integration Tests

Location: test/integration/

Test Suites: - providers_test.go - All LLM providers - tools_test.go - All tools end-to-end - workflows_test.go - Plan Mode, autonomy - features_test.go - Advanced features

Performance Tests

Location: test/performance/

Test Suites: - `codebase_mapping_bench_test.go` - Benchmark tree-sitter - `compression_bench_test.go` - Benchmark compression - `cache_bench_test.go` - Benchmark caching

End-to-End Tests

Location: `test/e2e/`

Scenarios: - Complete development workflow - Multi-provider failover - Large codebase handling - Browser automation - Voice input

Documentation Standards

Code Comments

- Godoc for all exported functions
- Package-level documentation
- Example code in comments

Markdown Documentation

- Clear structure with TOC
- Code examples for all features
- Screenshots/diagrams where helpful
- Links to related docs
- Version compatibility notes

README Updates

Sections to Add: - New provider list (14+ providers) - New tools list - Plan Mode section - Browser control section - Codebase mapping section - Link to comprehensive docs

Quality Gates

Before Merging Each Feature:

- All tests passing
- Coverage > 85%
- Documentation complete
- Examples working
- No linter errors

- Performance acceptable
- Security review passed

Before Release:

- All integration tests passing
 - E2E tests passing
 - User manual complete
 - API docs complete
 - Migration guide (if breaking changes)
 - Changelog updated
 - Version bumped
-

Parallelization Strategy

Week 1-2 (4 parallel tracks):

- **Track 1:** File system tools + Multi-file editing
- **Track 2:** Shell execution + Browser control
- **Track 3:** Codebase mapping
- **Track 4:** Plan Mode

Week 3-4 (4 parallel tracks):

- **Track 1:** Bedrock + Azure providers
- **Track 2:** VertexAI + Groq providers
- **Track 3:** Auto-commit + Context compression
- **Track 4:** Web tools + Tool confirmation

Week 5-6 (3 parallel tracks):

- **Track 1:** Voice-to-Code + Vision auto-switch
 - **Track 2:** Checkpoint snapshots + Autonomy modes
 - **Track 3:** Documentation + Testing
-

Risk Mitigation

Technical Risks:

1. **Tree-sitter complexity:** Use existing Aider patterns
2. **Browser control fragility:** Extensive error handling
3. **LLM API changes:** Version locking + adapters

4. **Performance issues:** Benchmarking + optimization

Project Risks:

1. **Scope creep:** Stick to defined features
 2. **Testing bottlenecks:** Parallelize test writing
 3. **Documentation debt:** Write docs alongside code
 4. **Integration issues:** Continuous integration testing
-

Success Metrics

Week 2 Checkpoint:

- 6 core tools implemented (file, shell, browser, mapping, edit, plan)
- 60+ unit tests passing
- Basic documentation complete

Week 4 Checkpoint:

- 4 new providers (Bedrock, Azure, VertexAI, Groq)
- 8 enhancement features (auto-commit, compression, web, confirmation)
- 100+ unit tests passing
- Integration tests passing

Week 6 Checkpoint:

- All advanced features (voice, snapshots, autonomy, vision)
- 150+ unit tests passing
- E2E tests passing
- Comprehensive documentation complete

Week 7 Final:

- All quality gates passed
 - User manual complete
 - API docs complete
 - Ready for release
-

Timeline Summary

Week	Focus	Deliverables
1-2	Core Tools	File, Shell, Browser, Mapping, Edit, Plan Mode
3-4	Providers & Enhancements	4 providers, Auto-commit, Compression, Web, Confirmation
5-6	Advanced Features	Voice, Snapshots, Autonomy, Vision
6-7	Documentation & Polish	Manual, API docs, E2E tests

Total: 7 weeks to full feature implementation

END OF IMPLEMENTATION PLAN